

PRUNING RULES FOR RUNWAY SEQUENCING

Simply put, the Runway Scheduling Problem is to find an optimal, ordered sequence of landings and take-offs for a collection of aircraft. This is constrained by maintaining asymmetric minimum separations between aircraft (due to wake turbulence), hard time windows and minimising CTOT violations. Solving this effectively is paramount to the economic and environmental efficiency of real-world airports. It is traditionally NP-hard, making a fast optimal algorithm for the general formulation likely unattainable.

Heathrow is a particularly challenging instance of this problem, with limited runway availability and intricate ground movement dynamics contributing to increased complexity. Currently, the TSAT/DMAN system deploys a branch-and-bound approach, supplemented by a set of powerful, manually inferred pruning rules that leverage structural features to considerably reduce the search space. Whereas heuristic approaches introduce complexity by adding meta-search layers atop the problem, pruning rules exploit key domain characteristics to simplify the problem without changing it, constituting a fundamentally new paradigm of optimization.

1 PROBLEM DESCRIPTION

Formally defined in Equation 1, the Runway Sequencing Problem is to determine a feasible take-off sequence for a set of aircraft, subject to time-slot and wake constraints. In the model from [1], the objective is to get all aircraft into the air as quickly as possible, while respecting their CTOT slots and avoiding disproportionate delay to any individual aircraft.

$$\mathcal{P} = (S, \delta, \langle ec, lc \rangle, \langle et, lt \rangle, b, c) \quad (1)$$

- S denotes the set of all aircraft in the problem.
- δ denotes a domain-complete function returning the minimum wake separation between 2 aircraft.
- $\langle ec, lc \rangle$ denote the CTOT slot of an aircraft, i.e. ec_i denotes the earliest CTOT time of aircraft i
- $\langle et, lt \rangle$ denote the hard time window of an aircraft, i.e. et_i denotes the start of the time window for aircraft i
- b denotes base-time, b_i is the earliest time that aircraft i can join the queue of aircraft waiting at the runway.
- c denotes the minimum time to reach the start of the queue and line up with the runway.

Assuming each aircraft will takeoff as early as possible (a valid assumption at busy airports), take-off time, denoted in Equation 3, is the latest of the release time or minimum wake separation takeoff (for all aircraft scheduled before i). Release time, Equation 2, denotes the earliest (possible) time an aircraft can be sequenced.

$$r_i = \max(b_i + c_i, et_i, ec_i) \quad (2)$$

$$t_i = \max\left(r_i, \max_{x \in s_i}(t_x + \delta_{xi})\right) \quad (3)$$

Orderings are ranked are using the multi-objective function in Equation 4. Firstly, we want to minimise *makespan*, that is the take-off time of the final aircraft in the sequence or total time to sequence all aircraft in S . In the second part, we consider the total delay, meaning the time-difference between an aircraft's base time and it's scheduled takeoff in the sequence, with a nonlinear scalar to promote equity. Here, we also consider costs relating to CTOT (calculated time-of-takeoff) violations, described straightforwardly in the definition of C .

$$F(s) = \left(\max_{i \in s} t_i, \sum_{i \in s} (W_1(t_i - b_i)^\alpha + W_2 C(t_i, lc_i)) \right) \quad (4)$$

$$C(t_i, lc_i) = \begin{cases} (0, t_i \leq lc_i) \\ (\omega_1(t_i - lc_i) + \omega_2, lc_i < t_i \leq lc_i + 300) \\ (\omega_3(t_i - lc_i) + \omega_4, t_i > lc_i + 300) \end{cases} \quad (5)$$

Additionally, Equation 4 and Equation 5 include weightings which operators can tune to prioritise different parts of the objective function. Several variables in the problem formulation reference points in time, or a period of time; formally, these are constrained as non-negative in Equation 6. That is, base times, queue times, CTOT-slots, time windows and wake separations are explicitly constrained to be ≥ 0 .

$$b_i, c_i, et_i, lt_i, ec_i, et_i \geq 0, \forall i \in S \quad \delta_{ij} \geq 0, \forall (i, j) \in S \times S \quad (6)$$

2 OPTIMAL RUNWAY SEQUENCING

2.1 BRUH MOMENTO

3 INTERVALS & CONTRACTORS

3.1 INFERENCE RULES FOR INTERVALS

$$\frac{x \in [a, b] \quad y \in [c, d]}{x + y \in [a + c, b + d]} \text{ [ADD-FORWARD]} \qquad \frac{x \in [a, b] \quad y \in [c, d]}{x - y \in [a - d, b - c]} \text{ [SUB-FORWARD]} \quad (7)$$

$$\frac{x \in [a, b] \quad y \in [c, d]}{x \times y \in [\min(ac, ad, bc, bd), \max(ac, ad, bc, bd)]} \text{ [MULT-FORWARD]} \quad (8)$$

$$\frac{x \in [a, b] \quad y \in [c, d] \quad 0 \notin [c, d]}{\frac{x}{y} \in [a, b] \times [\frac{1}{d}, \frac{1}{c}]} \text{ [DIV-FORWARD]} \quad (9)$$

4 SMT

Runway Sequencing consists of integer and real arithmetic, piecewise-linear constraints, max operators, polynomial functions, and finite sets of symbolic aircraft; consequently, verification of pruning rules sits in quantifier-free non-linear real arithmetic (QF_NRA). Currently, each pruning rule has been proved using pen-and-paper, which is difficult to reliably construct and impossible to automate, [1].

Pruning rules in [1] are expressed as universally quantified statements over all combinations of sequences of aircraft, subject to some precondition. Even if all parameters are artificially restricted to finite ranges, a brute-force enumeration to verify a rule still suffers from combinatorial explosion; the number of possible instances and orderings increases super-exponentially with the number of aircraft. This makes exhaustive verification computationally infeasible beyond very small toy problems, therefore unsuitable to verify in real-world cases.

Given the high-impact, safety critical problem domain, establishing formal correctness is necessary; holding statistical confidence that a rule holds simply isn't enough. As a result, sampling-based simulation/fuzz testing is inadequate for the verification of pruning rules, since such methodologies only prove the presence of counterexamples, not their absence [2].

Establishing this formal correctness necessitates the symbolic quantification over both $\mathbb{R}$ and $\mathbb{Z}$; thus any approach utilizing integer programming techniques can not faithfully encode the problem. However, integer programming techniques could be utilized for the operational validation of pruning rules, but are unsuitable for universal, mathematical proof in the general case.

Recreating the problem and pruning rules in a higher-order logic or dependently typed programming language and proving interactively could guarantee formal correctness and produce a set of extremely strong, foundational proofs. However, similar to pen-and-paper, this requires strong user-guidance and singular proofs are often hundreds of lines. There are limited methodologies in the automated synthesis of proofs under this methodology.

On the other hand, SMT supports symbolic parameters over $\mathbb{Z}$ and $\mathbb{R}$, reasons exactly over max, conditional logic and piecewise-linear penalties. It can provably negate the existence of a counterexample, or provide (useful) concrete counterexamples if one exists. Moreover, the methodology runs automatically, without the need for weeks of specialist, theorem-prover development. One could place it as a Goldilocks methodology, sitting between operational verification in IP and the full, mathematical rigour of a theorem proving system such as HOL or Lean4.

However, there are limitations: proofs in SMT are large, difficult to inspect and not directly interpretable by humans. Thus, the trust chain depends on the correctness of the solver, correctness of the theory solver, and absence of internal bugs. The methodology should be combined with independent solver cross-checks, or even certificate verification in a stronger logic such as HOL in some cases (not non-linear real arithmetic).

4.1 IS IT PROOF?

Some would argue that Mathematics requires transparent deductive reasoning; whereas SMT provides opaque algorithmic answers.

5 RULEFORMS

Let $\mathbb{F}$ denote the set of *rule-forms*, which act as syntactic templates describing the structure in which a pruning rule operates. Forms define the activation environment in which conditions to prune search branches can arise, the symbols a pruning condition of that form can reference/include (detailed later in Section 6.1) and some post-condition function $\mathcal{A}$.

$$F := \langle \mathcal{X}, \Phi_F(\sigma), \mathcal{A}(\sigma) \rangle \quad \forall F \in \mathbb{F} \quad (10)$$

- $\mathcal{X}$ denotes the set of variable bundles, where each element $\sigma \in \mathcal{X}$ contains a full valuation of all variables.
- $\Phi_F(\sigma)$ denotes the *dependent conjunction of all feasibility constraints* on some state $\sigma \in \mathcal{X}$.
- $\mathcal{A}(\sigma)$ denotes the *acceptance function*, for some pruning condition $\beta : \mathcal{X} \rightarrow \mathbb{B}$.

One example is the *complete order form*, which states that under some precondition R , sequencing aircraft i before j in an arbitrary sequence can never yield a higher objective cost than the swapped ordering $j \prec i$. Explicitly, *complete order form* defines two arbitrary sequences (S, S') where $i \prec j$ and $j \prec i$, with a post-condition necessitating: $F(S) \leq F(S')$. Inside of this form, any *sound pruning condition*, would necessitate that $F(S) \leq F(S')$.

6 PRUNING CONDITION SYNTAX

Pruning conditions are expressed in the language defined in Grammar 1. Syntactic typing is enforced by an ordered, multi-sorted grammar, ensuring that every well-formed rule denotes a Boolean expression: true when the rule holds and false otherwise. Grammar 1 describes pruning conditions in their canonical form, without syntactic-sugaring.

Boolean Term	β	$\beta \wedge \beta$	Conjunction
		$\beta \vee \beta$	Disjunction
		$\neg \beta$	Negation
		$\alpha \rho \alpha$	Comparison
Arithmetic Expression	α	$\alpha \oplus \alpha_t$	Additive Expression
		α_t	Arithmetic Term
Arithmetic Term	α_t	$\alpha_t \otimes \alpha_f$	Multiplicative Expression
		α_f	Arithmetic Factor
Arithmetic Factor	α_f	$\mathbb{Z}$	Numerical
		$\mathbb{S}(F)$	Symbol
		(α)	Arithmetic Expression
Comparison Operator	ρ	$>$	Greater Than
		$<$	Less Than
		$=$	Equality
		$\geq$	Greater Than or Equal To
		$\leq$	Less Than or Equal To
Additive Expression	$\oplus$	$+$	Addition
		$-$	Subtraction
Multiplicative Expression	$\otimes$	$\times$	Multiplication
		$\div$	Division

Grammar 1: Correct by Construction Language for Pruning Rules

6.1 SYMBOL SETS

Introduced in Grammar 1, is a symbol-set operator $\mathbb{S}$, which assigns to each *rule form* $F \in \mathbb{F}$ a set of admissible symbols. Rules may reference only symbols contained in their symbol-sets; references to symbols outside $\mathbb{S}(F)$ are ill-formed. The collection $\mathbb{S}(F)$ is therefore an indexed family of symbol-sets (over index $\mathbb{F}$), or $F \in \mathbb{F}, F \vdash \mathbb{S}(F)$. The symbol set for a given form, or $\mathbb{S}(F)$, is defined as the least set of symbols satisfying the following closures¹:

- All *functional attributes* lifted over *all aircraft*, or $f_p \in \mathbb{S}(F), \forall f \in \{r, c, b, et, lt, ec, lc, t, t'\}, \forall_p \in \mathcal{AC}(F)$
- All *wake separation* pairings over *all aircraft*, or $\delta_{ij} \forall \langle i, j \rangle \in \mathcal{AC}(F) \times \mathcal{AC}(F)$

Intuitively, these rules enforce that a pruning rule may only reference relevant symbols in its form definition. Aircraft variables themselves are explicitly excluded from symbol sets $a \notin \mathbb{S}(F), \forall a \in \mathcal{AC}(F)$, since these denote arbitrary, unique identifiers with no semantic relevance beyond indexing functional attributes (such as *release times*).

¹We assume that $\mathcal{AC} : \mathbb{F} \rightarrow \mathcal{S}$ is a function extracting the set of aircraft placeholders appearing syntactically in a given rule-form, that is for every $F \in \mathbb{F}, \mathcal{AC}(F) = \{a \mid a \text{ is an individual aircraft variable bound in form } F\}$.

7 PRUNING CONDITION SEMANTICS & VALIDITY

We formally specify the semantic obligations (*sound* and *applicable*) that a *condition, rule-form* pair must meet to be considered *valid* within the restrictive pruning framework. These conditions are provided under the context of an SMT-based verification methodology.

7.1 CONDITION SOUNDNESS

Some pruning rule β is *sound* for form F if whenever the rule is triggered in a feasible configuration, the dominance relation (post-condition) necessitated by the form always holds. That is, for every configuration $\sigma \in \mathcal{X}$, that is feasible under the form constraints $\Phi_F(\sigma)$ and satisfies the pruning condition $\beta(\sigma)$, the dominance post-condition required by the form must hold; equivalently the acceptance predicate $\mathcal{A}(\sigma)$ must hold true, Equation 11.

$$\forall \sigma \in \mathcal{X}, (\Phi_F(\sigma) \wedge \beta(\sigma) \implies \mathcal{A}(\sigma)) \quad (11)$$

7.2 VACUOUSLY UNSATISFIABLE / MIN-APPLICABILITY

Consider $\delta_{ij} \neq \delta_{ij}$ as a candidate condition, it is syntactically well-formed, and reduces to a boolean yet semantically it is vacuously false. However, under a naïve SMT-interpretation of pruning rule validity– to be *unsatisfiable* is to be *verified*, yet this is not; consequently we must additionally rule out vacuous predicates whose truth set is empty.

This is relevant, because if the sole priority is the context-blind pursuit of unsatisfiability, it's far easier to construct a vacuously false case than an effective pruning rule. To eliminate these common cases, we must obtain a witness of applicability, that is an example that the rule can hold true in a feasible scenario. That is, we must verify the *domain* of the pruning condition to be non-empty within the form constraints, Equation 12.

$$\exists \sigma \in \mathcal{X}, (\Phi_F(\sigma) \wedge \beta(\sigma)) \quad (12)$$

7.3 CONDITION COMPLETENESS (NON-REQUISITE)

Pruning conditions are *complete* within their form if they trigger for all feasible configurations in which the dominance post-condition is true, thereby capturing the full domain where pruning would be correct, Equation 13. However, for a *pruning rule* to be valid, it is not necessary to be *complete*, as the objective is merely to discard some dominated configurations, as opposed to characterising the entire dominance region within the search space.

$$\forall \sigma \in \mathcal{X}, (\Phi_F(\sigma) \wedge \mathcal{A}(\sigma) \implies \beta(\sigma)) \quad (13)$$

7.4 CONDITION NON-REDUNDANCY

Pruning conditions can be considered *non-redundant* if they are not logically implied by existing problem constraints or pre-defined pruning rules; that is, there exists a feasible configuration σ , where candidate rule β' holds at least one existing rule β does not. *Non-redundancy* is desirable for efficacy but not necessary for a rule to be valid.

$$\forall \beta \in \beta^*, \exists \sigma \in \mathcal{X}, (\Phi_F(\sigma) \wedge \beta'(\sigma) \wedge \neg \beta(\sigma)) \quad (14)$$

7.5 CONDITION APPLICABILITY (EXACT AND APPROXIMATE)

Condition applicability quantifies how often a pruning condition β activates over feasible configurations within its rule form F . Computing this fraction exactly (see p_F) requires model counting, which is computationally intractable in general and may be undefined when the state space is infinite/enumerable.

We approximate applicability using Monte Carlo sampling, drawing configurations independently from a uniform product distribution over bounded variable domains. Specifically, $\hat{p}_F^{(n)}$ is defined as a Monte Carlo proxy of p_F obtained by evaluating β on sample configurations of $\mathcal{X}$ and computing the empirical activations as a fraction of the feasible samples.

$$p_F(\beta) = \frac{\sum_{\sigma \in \mathcal{X}} 1[\Phi_F(\sigma) \wedge \beta(\sigma)]}{\sum_{\sigma \in \mathcal{X}} 1[\Phi_F(\sigma)]} \quad \hat{p}_F^{(n)}(\beta) = \frac{\sum_{k=1}^n 1[\Phi_F(\sigma_k) \wedge \beta(\sigma_k)]}{\sum_{k=1}^n 1[\Phi_F(\sigma_k)]}, \quad \sigma_1 \dots \sigma_n \sim \mathcal{X} \quad (15)$$

Applicability induces an extensional preorder over pruning conditions under form F , ordered by frequency of activation, with $\top$ denoting conditions that apply on all feasible configurations and $\perp$ denoting conditions that never activate.

Formally, this preorder can be logically defined as: $\beta_1 \preceq_{p_F} \beta_2 \stackrel{\text{def}}{\iff} p_F(\beta_1) \leq p_F(\beta_2)$.

7.6 CONDITION EQUIVALENCE AND FORM-EQUIVALENCE

Logical equivalence between two pruning conditions can be established by proving their bi-implication, however; two conditions β, β' can be considered equivalent under their *form* F , if for all feasible configurations in $\mathcal{X}$, their bi-implication holds, i.e. they only differ semantically outside of the feasible region. Whilst weaker than traditional equivalence, form-equivalence ignores irrelevant distinctions outside the form constraints.

$$\beta \stackrel{\text{def}}{=} \beta' \iff \forall \sigma \in \mathcal{X}, (\Phi_F(\sigma) \implies (\beta(\sigma) \iff \beta'(\sigma))) \quad (16)$$

8 ALGEBRAIC STRUCTURE OF PRUNING CONDITIONS

Having defined the syntax and semantic validity of pruning conditions, we now characterise the algebraic structure they induce under semantic implication. This structure formalises notions of strengthening and weakening, and explains the monotonic propagation of soundness as well as its role in guiding automated pruning-condition synthesis.

8.1 STRENGTHENING AND WEAKENING

Let $\beta, \beta' : \mathcal{X} \rightarrow \mathbb{B}$ be pruning conditions of rule-form F . It is said that β *strengthens* (or *refines*) β' , if for all feasible configurations, σ , in $\mathcal{X}$, it holds that $\beta(\sigma) \Rightarrow \beta'(\sigma)$. Dually, β' *weakens* (or *generalizes*) β if for all feasible configurations, σ , $\beta'(\sigma) \Rightarrow \beta(\sigma)$. Intuitively, if some pruning condition β is stronger than another condition, β' , then β applies/activates over less configurations in $\mathcal{X}$ and vice versa.

$$\begin{aligned} \beta \text{ strengthens } \beta' & \text{ iff } \forall \sigma \in \mathcal{X}, (\Phi_F(\sigma) \Rightarrow (\beta(\sigma) \Rightarrow \beta'(\sigma))) \\ \beta \text{ weakens } \beta' & \text{ iff } \forall \sigma \in \mathcal{X}, (\Phi_F(\sigma) \Rightarrow (\beta'(\sigma) \Rightarrow \beta(\sigma))) \end{aligned} \quad (17)$$

Strengthening induces a preorder over valid pruning conditions, since logical implication is reflexive and transitive but antisymmetry holds only up to semantic equivalence. We introduce this structure here primarily to reason about dominance; its fuller algorithmic significance becomes apparent when the preorder is later lifted to a lattice.

Within this preorder, soundness properties propagate monotonically. Importantly, *unsoundness* is monotone in respect to weakening, if a pruning condition β is *unsound*, then any weakening of β is also unsound, since it applies to a superset of configurations. Dually, *soundness* is monotone to strengthening, if β is sound, then any strengthening of β is also sound, since it applies within a subset of sound configurations.

8.2 LINDENBAUM-TARSKI ALGEBRA & QUOTIENTING

Whilst *strengthening/weakening* alone induces a preorder defined by semantic implication, it is not antisymmetric: syntactically distinct pruning conditions may be semantically equivalent. To obtain a genuine partial order - and to reason algebraically about the space of pruning conditions rather than individual syntactic representations, the set of pruning conditions is quotiented by semantic equivalence/bi-implication.

The resulting quotient structure is the Lindenbaum-Tarski algebra; its elements are equivalence classes of pruning conditions, ordered by entailment, with conjunction and disjunction inducing meet and join operations. This algebraic view makes the search regions eliminated by unsoundness-weakening monotonicity precise as upward and downward closures in the induced lattice². Formally, $[\beta] \leq [\beta']$ iff $\beta' \models \beta$ for all feasible σ in $\mathcal{X}$.

Figure 1 depicts a finite sublattice of the infinite Lindenbaum-Tarski lattice, where $[\beta]$ nodes denote equivalence classes of pruning conditions ordered by semantic entailment; vertical position corresponds to weakening/strengthening, while horizontal placement is purely for layout. Here, subscript indicies denote relative lattice depth with respect to reference element $[\beta_0]$; primes distinguish incomparable elements at the same lattice level, but convey no semantic meaning.

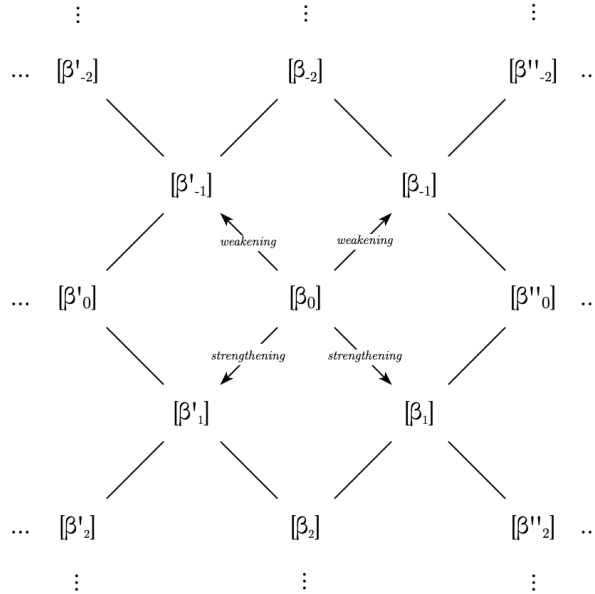


Figure 1: Lindenbaum-Tarski Lattice of Pruning Conditions

²Partially ordered sets are termed *lattices* when every pair of elements admits both a greatest lower bound (meet) and a least upper bound (join), which in the present setting correspond to conjunction, $\wedge$, and disjunction, $\vee$, respectively.

8.3 MONOTONICITY OF SOUNDNESS

Figure 2 depicts the monotonic propagation of unsoundness under weakening; if an equivalence class of pruning conditions $[\beta_0]$ is proven unsound, then any weakening $[\beta_0^\downarrow]$ is also unsound. The shaded region therefore denotes the upward closure of $[\beta_0]$, which can be discarded from consideration once a single feasible counterexample is identified.

Dually, Figure 3 illustrates that soundness is monotone under strengthening: if $[\beta_0]$ is sound, then every $[\beta_n] \leq [\beta_0]$ is sound. The shaded region denotes the downward closure of $[\beta_0]$, certifying all refinements $[\beta_0^\uparrow]$ without additional proof. Once a pruning condition is proven sound, further *strengthening* does not increase correctness, only restricts applicability.

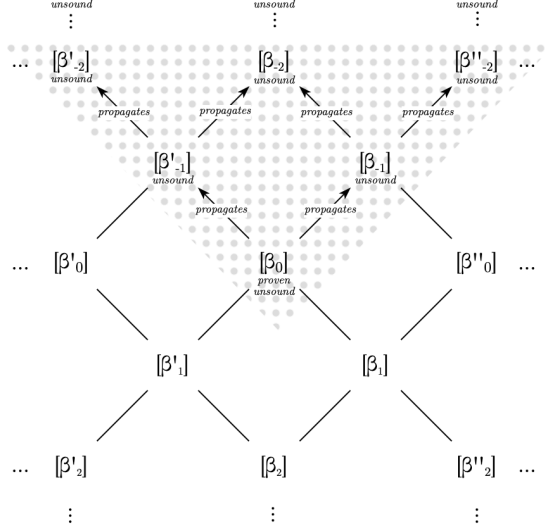


Figure 2: Unsoundness Monotonicity Closure

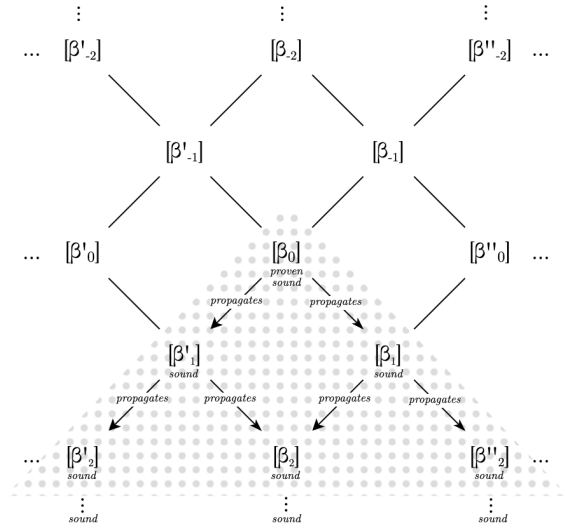


Figure 3: Soundness Monotonicity Closure

Under a naïve synthesis procedure, a singular counterexample refutes a singular candidate condition; however, under the lattice interpretation, all weakenings of the condition are refuted, since they apply to a superset of configurations that include the violating instance. Consequently, synthesis methodologies are able to prune the entire upward closure of that condition in the lattice, which typically contains an exponentially large family of syntactically distinct candidates.

Operationally, synthesis seeks pruning conditions of maximal applicability subject to soundness. Accordingly, once a condition is established as sound, it is natural to consider its successive weakenings, generalising until a fixed point is reached beyond which soundness no longer holds, containing the maximal sound element in that subregion.

9 SYNTHETISING PRUNING CONDITIONS

Having defined the rule language and its semantic obligations, we now turn to the problem of automatically synthesising pruning conditions that satisfy these constraints. Here, we treat pruning conditions as objects of program synthesis whose soundness must hold for every instantiation of their rule-form.

However, even under the restrictive language, the space of boolean-expressions over symbol sets is exponential in the number of symbols, depth of terms and combination of operators. Most randomly-constructed rules are semantically meaningless, vacuously unsatisfiable, or incompatible with form constraints, making a naïve enumeration hopeless.

Moreover, valid pruning conditions must satisfy a universally quantified correctness obligation, which makes each verification query expensive and necessitates a synthesis approach that minimises such queries while still exploring a large, highly non-convex, disconnected search space.

9.1 COUNTER-EXAMPLE GUIDED INDUCTIVE SYNTHESIS (CEGIS)

Evolutionary search depends on a graded notion of correctness; candidate programs must be rewarded for satisfying portions of their specification so that mutation has a meaningful search direction. However, universal proof is binary, and therefore intrinsically non-graded – collapsing the fitness landscape and making incremental progression non-trivial.

CEGIS avoids this by replacing the necessity for universal proof with a growing set of finite examples extracted from the SMT-solver at each generation, [3]. Intuitively, this relies on the assumption that programs that work for n examples have a higher-probability of working for $n + 1$ examples, since they have likely internalised some of the specification structure.

As more counterexamples are discovered, the hypothesis space shrinks, and once no counterexample exists - the candidate program becomes universally sound, by virtue of the SMT verification. This process transforms universal constraint into a sequence of tractable problems with smoother fitness landscapes.

9.1.1 Vacuous Satisfaction & Activation Region

In the context of counterexample-driven pruning-condition synthesis, counterexamples are instances in which activation would lead to pruning the better off solution. Given this, any system implementing this alone would punish unsound activations but never reward those that are sound; by extension, the optimal behavior of a condition becomes to never activate at all.

Such rules trivially avoid penalty, but offer little in the way of pruning power. In order to combat this downwards pressure in the preorder (*towards* $\perp$), the fitness function must reward those rules with a large activation region, or conditions that hold true much of the time. It is possible to estimate the activation percentage of a pruning rule using *model counting* - that is calculating the number of total states where some condition β holds under Φ_F , divided by the total number of states where Φ_F holds.

Unfortunately, accurate *model counting* is of exponential time complexity, but it is possible to quickly generate an approximation through Monte-Carlo sampling over the possible state space. This provides *upwards pressure* (*towards* $\top$), rewarding pruning conditions that are more applicable, and promoting the ingestion of the problem structure.

9.2 OBJECTIVE FUNCTION

Candidate pruning rules are assigned a multi-objective fitness through the following function, $F(\beta)$

$$F(\beta) = \frac{|\Sigma^*(\beta)|}{|\Sigma^*|} + \left(1 - \frac{|\beta|}{\max_{i \in P} |\beta_i|}\right) + H\left(\frac{\mathcal{M}(\beta)}{|\mathcal{M}|}\right) \quad (18)$$

$$H(p) = -p \log p - (1 - p) \log(1 - p) \quad (19)$$

In this definition,

- Σ^* denotes the set of counterexamples, with $\Sigma^*(\beta) \subseteq \Sigma^*$ denoting the subset where β is activated.
- $|\beta|$ denotes the syntactic size of β , normalised by the largest rule in the current population P .
- $\mathcal{M}$ is a Monte-Carlo sample of feasible configurations, and $\mathcal{M}(\beta) \subseteq \mathcal{M}$ the samples where β activates.

9.3 GENERALISATION VS. REFINEMENT ON MUTATIONS

- Sound pruning conditions should be able to mutate *up* (become more general)
- Unsound pruning conditions should only be able to mutate *down* (become more refined)

9.4 CANDIDATE EQUIVALENCE

Duplicate candidates within genetic populations are detrimental to search efficiency, as they reduce effective diversity without contributing new behavior. In addition to syntactically equivalent candidates, semantically equivalent candidates arise frequently, owing to the domain's rich algebraic structure. Establishing semantic distinctness among candidates via symbolic methods is computationally expensive.

Mitigating this cost, Monte Carlo and Σ^* (counterexample set) activation signatures are used to induce an over-approximation of semantic equivalence classes: candidates assigned to different classes are guaranteed to be semantically distinct, while candidates within the same class are subjected to exact verification. Semantic equivalence is established by querying an SMT solver for a counterexample to bi-implication between candidates.

BIBLIOGRAPHY

- [1] G. De Maere, J. A. Atkin, and E. K. Burke, "Pruning rules for optimal runway sequencing," *Transportation Science*, vol. 52, no. 4, pp. 898–916, 2018.
- [2] E. W. Dijkstra, "On the reliability of programs," *Edsger Wybe Dijkstra: His Life, Work, and Legacy*. pp. 359–370, 2022.
- [3] C. David and D. Kroening, "Program synthesis: challenges and opportunities," *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 375, no. 2104, p. 20150403, Oct. 2017, doi: [10.1098/rsta.2015.0403](https://doi.org/10.1098/rsta.2015.0403).